

OPTION B INSTITUTE FAMILY

M11 - STORAGE & FILE FORMATS FOUNDATIONS

Standalone Zero-to-Job-Ready Data Engineer 2026

BEGINNER-FIRST MASTER REPAIR RC4

10 canonical lessons | full 16-stage teacher contract | protected attempt-first
review

This learner book is rebuilt from the frozen canonical technical material. It is not a front-matter patch: every lesson is re-laid out into the shared institute teaching grammar while preserving the course competency, source route, guided work, independent proof and validation boundary.

This book is your teacher

Use the book in order. External videos and official docs are visual companions, not hidden prerequisites. Watch or read only the assigned scope, then close it before the independent task.

The 16-stage lesson loop

Why it matters -> What you learn -> Prerequisite -> Picture it -> New words -> Useful source -> Follow once -> Expected result -> Why it worked -> Beginner mistakes -> Recovery -> Independent task -> Validate -> Teach it back -> Protected review + changed retry -> Evidence/mastery + exact next route.

Attempt-first protection

Never open a protected lesson review before saving a genuine independent attempt. If repair is needed, compare only the weak idea, close the review, solve a changed retry, then validate again. A solution seen during review is repair evidence, not mastery.

What counts as mastery

A reproducible artifact or explanation, an independent validation control, preserved failure/recovery evidence, a changed transfer task completed without the worked answer open, and an own-words explanation. Percent-complete is not a substitute.

How to use your devices

Phone: assigned video/reference, reading, revision. Laptop/PC: every real command, query, file, workbook, code task, validation and evidence artifact. Install only what the current lesson asks for.

Contents / Index

This book is your teacher	2
Contents / Index	3
Prerequisite / tool map	4
Practice / review / Gate route	4
DE11-01 - Row vs columnar storage; where Parquet fits	5
DE11-02 - Parquet schema, row groups and metadata	7
DE11-03 - Projection, predicates and analytical reads	9
DE11-04 - Compression, small files and compaction	11
DE11-05 - Partitioning strategy and over-partitioning	13
DE11-06 - Lake vs warehouse vs lakehouse	15
DE11-07 - Delta tables, transaction log and ACID concepts	17
DE11-08 - MERGE/upsert and current-state records	19
DE11-09 - Schema enforcement, controlled evolution and compatibility	21
DE11-10 - Time travel, retention, VACUUM safety and medallion checkpoint	23
Module mastery checklist + revision quick reference	25

Prerequisite / tool map

Choose storage/file formats and reason about Parquet/Delta foundations, partitioning, schema and time travel.

Item	Rule
Entry prerequisite	Previous module/Gate evidence must be stable.
Primary device	PC/laptop for hands-on work; phone is support only.
External sources	Optional/supporting unless the lesson explicitly marks an official current reference.
Evidence	Module practical + fresh retest + saved evidence; no critical unresolved failure.

Practice / review / Gate route

Lesson teaching -> follow once -> independent attempt -> validate -> save evidence. Only after the genuine attempt may protected review open. If weak: targeted repair -> close review -> changed retry -> revalidate. Finish the module practical/retest route before any downstream Gate that depends on this module.

DE11-01 - Row vs columnar storage; where Parquet fits

1. Why it matters

This matters because later Storage & File Formats Foundations work assumes this competency is reliable, not merely familiar. You must be able to reproduce it and validate the result without copying a finished answer.

2. What you are learning

Explain why analytical systems prefer columnar storage, identify where Parquet fits, and prove the difference between logical table grain and physical file layout.

3. Prerequisite / starting state

Starting state: no prior technical knowledge is assumed beyond the module Start Here instructions.

4. Picture it in your head

A table is a logical collection of rows, but a storage format decides how those values are laid out on disk. Row-oriented layouts keep the fields for one record near each other, which is convenient when applications frequently read or update complete records.

5. New words before we use them

A table is a logical collection of rows, but a storage format decides how those values are laid out on disk. Row-oriented layouts keep the fields for one record near each other, which is convenient when applications frequently read or update complete records. Columnar formats group values from the same column together.

6. Useful video / source

Source: Databricks Academy — free data/lakehouse training

Assigned scope: Choose the matching storage / Parquet / Delta module assigned by Mentor

How to use it: Use the matching free training unit for the visual explanation, then return to the Teaching Book and execute the local course lab.

[Open source](#)

7. Follow with me once

Use orders.csv. State its grain: one row per order. Compare a question that needs every field for O1008 with a question that needs only region and amount for all orders. The first resembles row retrieval; the second is the kind of scan where columnar storage can help. Run `parquet_probe.py` on the bundled Apache Parquet fixture. Confirm PAR1 at both ends. Then install PyArrow 25.0.* on your learner machine and rerun to expose schema/row-group metadata.

8. Expected result / what you should see

Your explanation must not call Parquet a database. ``orders.csv`` has 12 rows, quantity 24, amount 2780.00, four regions and four dates.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

If row-vs-columnar still feels abstract, draw six columns and four rows on paper, then circle the values needed for ``SUM(amount) GROUP BY region``. Count how many cells are relevant.

11. If you get stuck

If row-vs-columnar still feels abstract, draw six columns and four rows on paper, then circle the values needed for ``SUM(amount) GROUP BY region``. Count how many cells are relevant.

12. Small independent task

Write a five-sentence explanation that separates logical row grain, columnar storage, Parquet, a query engine, and Delta Lake. Use one concrete analytics query from orders.csv.

13. Validate

Your explanation must not call Parquet a database. `orders.csv` has 12 rows, quantity 24, amount 2780.00, four regions and four dates.

14. Teach it back

Why can the same logical table be stored in CSV, Parquet, or a database table without changing its business grain?

15. Protected review + changed retry

Only after the independent attempt and evidence are saved may you open the matching M11 Review/Repair section. Close it before a changed retry.

16. Evidence / mastery + exact next route

Save the command/code or decision artifact, observed output/control, one error or uncertainty, the repair you made, and a 60-90 second explain-back. Do not mark mastered from reading alone. Exact next route: DE11-02

DE11-02 - Parquet schema, row groups and metadata

1. Why it matters

This matters because later Storage & File Formats Foundations work assumes this competency is reliable, not merely familiar. You must be able to reproduce it and validate the result without copying a finished answer.

2. What you are learning

Read Parquet schema/metadata and explain row groups, column chunks, pages and the footer without confusing them with business partitions.

3. Prerequisite / starting state

Prerequisite: DE11-01 mastered with saved evidence.

4. Picture it in your head

A Parquet file is not one undifferentiated block. Rows are divided into row groups.

5. New words before we use them

A Parquet file is not one undifferentiated block. Rows are divided into row groups. Within a row group, each column is stored as a column chunk; chunks contain encoded/compressed pages.

6. Useful video / source

Source: Databricks Academy — free data/lakehouse training

Assigned scope: Choose the matching storage / Parquet / Delta module assigned by Mentor

How to use it: Use the matching free training unit for the visual explanation, then return to the Teaching Book and execute the local course lab.

[Open source](#)

7. Follow with me once

The pure-Python probe reads the final four-byte PAR1 marker and the four-byte little-endian footer-length field immediately before it. It does not decode Thrift metadata; with PyArrow installed, ParquetFile exposes row-group count, total rows and schema safely. Use the bundled Apache fixture. Record file size, footer metadata length, row-group count and schema when PyArrow is available. Point to the difference between physical row groups and your region directories from the layout lab.

8. Expected result / what you should see

For 12 rows written with a row-group target of 4, expect 3 row groups in this small controlled file. Do not generalize this exact count to production writers.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

If PyArrow import fails, verify the active interpreter and run ``python -m pip install "pyarrow==25.0.*"``; use the PAR1 probe meanwhile. Do not install Spark to solve a PyArrow problem.

11. If you get stuck

If PyArrow import fails, verify the active interpreter and run ``python -m pip install "pyarrow==25.0.*"``; use the PAR1 probe meanwhile. Do not install Spark to solve a PyArrow problem.

12. Small independent task

Create ``orders_from_course.parquet`` using `build_course_parquet.py`, which writes `row_group_size=4`. Predict the approximate row-group count before running, then verify it.

13. Validate

For 12 rows written with a row-group target of 4, expect 3 row groups in this small controlled file. Do not generalize this exact count to production writers.

14. Teach it back

What is the difference between a row group and a partition directory, and why does Parquet keep metadata in a footer?

15. Protected review + changed retry

Only after the independent attempt and evidence are saved may you open the matching M11 Review/Repair section. Close it before a changed retry.

16. Evidence / mastery + exact next route

Save the command/code or decision artifact, observed output/control, one error or uncertainty, the repair you made, and a 60-90 second explain-back. Do not mark mastered from reading alone. Exact next route: DE11-03

DE11-03 - Projection, predicates and analytical reads

1. Why it matters

This matters because later Storage & File Formats Foundations work assumes this competency is reliable, not merely familiar. You must be able to reproduce it and validate the result without copying a finished answer.

2. What you are learning

Use projection and predicates deliberately, and explain how metadata/statistics may reduce physical reading without promising impossible zero-I/O behavior.

3. Prerequisite / starting state

Prerequisite: DE11-02 mastered with saved evidence.

4. Picture it in your head

Projection means requesting only the columns a query needs. Predicate filtering means expressing row conditions such as ``region` = East`` or ``amount` >= 250``.

5. New words before we use them

Projection means requesting only the columns a query needs. Predicate filtering means expressing row conditions such as ``region` = East`` or ``amount` >= 250``. Columnar storage makes projection powerful because an engine can avoid decoding unrelated columns.

6. Useful video / source

Source: Databricks Academy — free data/lakehouse training

Assigned scope: Choose the matching storage / Parquet / Delta module assigned by Mentor

How to use it: Use the matching free training unit for the visual explanation, then return to the Teaching Book and execute the local course lab.

[Open source](#)

7. Follow with me once

With PyArrow, read only ``region`` and ``amount`` from the course Parquet, then filter `amount >= 250`. Compare the logical result with the same condition evaluated from `orders.csv`. Record selected columns, qualifying order IDs, qualifying row count and `sum(amount)`. Keep a separate full-scan calculation as a correctness control.

8. Expected result / what you should see

For `amount >= 250` on the course data, qualifying orders are O1002, O1004, O1008, O1010, O1012; count 5 and amount 1610.00.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

If a filtered read returns unexpected rows, prove the predicate against CSV first. Separate a logic error from a storage/engine optimization issue.

11. If you get stuck

If a filtered read returns unexpected rows, prove the predicate against CSV first. Separate a logic error from a storage/engine optimization issue.

12. Small independent task

Choose another two-column projection and one predicate. Explain which physical work an engine might skip and which result control proves correctness regardless of optimization.

13. Validate

For amount ≥ 250 on the course data, qualifying orders are O1002, O1004, O1008, O1010, O1012; count 5 and amount 1610.00.

14. Teach it back

Why is projection an I/O optimization while result reconciliation is a correctness check?

15. Protected review + changed retry

Only after the independent attempt and evidence are saved may you open the matching M11 Review/Repair section. Close it before a changed retry.

16. Evidence / mastery + exact next route

Save the command/code or decision artifact, observed output/control, one error or uncertainty, the repair you made, and a 60-90 second explain-back. Do not mark mastered from reading alone. Exact next route: DE11-04

DE11-04 - Compression, small files and compaction

1. Why it matters

This matters because later Storage & File Formats Foundations work assumes this competency is reliable, not merely familiar. You must be able to reproduce it and validate the result without copying a finished answer.

2. What you are learning

Explain compression, diagnose the small-file problem, and design compaction without confusing fewer files with fewer logical rows.

3. Prerequisite / starting state

Prerequisite: DE11-03 mastered with saved evidence.

4. Picture it in your head

Compression reduces bytes, while compaction reduces file count by combining many small files into fewer larger files. They are related but not identical.

5. New words before we use them

Compression reduces bytes, while compaction reduces file count by combining many small files into fewer larger files. They are related but not identical. Parquet commonly compresses within column chunks; repeated values often compress well.

6. Useful video / source

Source: Databricks Academy — free data/lakehouse training

Assigned scope: Choose the matching storage / Parquet / Delta module assigned by Mentor

How to use it: Use the matching free training unit for the visual explanation, then return to the Teaching Book and execute the local course lab.

[Open source](#)

7. Follow with me once

Run `layout_lab.py`. It deliberately creates 12 one-row CSV files and a single compacted.csv from the same 12 rows. This is a file-count demonstration only, not a claim that CSV compaction equals Delta OPTIMIZE. Reconcile tiny-file rows to compacted rows and the original controls. Then inspect Parquet compression choices in current PyArrow/Databricks references.

8. Expected result / what you should see

The classroom layout has 12 tiny files and 1 compacted file while preserving the 12-row business dataset. Normal amount remains 2780.00.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

If you think “one huge file is always best,” list concurrency, parallelism and rewrite-cost trade-offs. The goal is healthy file sizing, not minimum file count.

11. If you get stuck

If you think “one huge file is always best,” list concurrency, parallelism and rewrite-cost trade-offs. The goal is healthy file sizing, not minimum file count.

12. Small independent task

Write a production decision note: when would you compact, what would you measure, and what controls must remain invariant? Include one risk of compacting too aggressively.

13. Validate

The classroom layout has 12 tiny files and 1 compacted file while preserving the 12-row business dataset. Normal amount remains 2780.00.

14. Teach it back

What is the difference between compression and compaction, and why can 10,000 tiny compressed files still be a problem?

15. Protected review + changed retry

Only after the independent attempt and evidence are saved may you open the matching M11 Review/Repair section. Close it before a changed retry.

16. Evidence / mastery + exact next route

Save the command/code or decision artifact, observed output/control, one error or uncertainty, the repair you made, and a 60-90 second explain-back. Do not mark mastered from reading alone. Exact next route: DE11-05

DE11-05 - Partitioning strategy and over-partitioning

1. Why it matters

This matters because later Storage & File Formats Foundations work assumes this competency is reliable, not merely familiar. You must be able to reproduce it and validate the result without copying a finished answer.

2. What you are learning

Choose a partitioning strategy from query patterns and cardinality, and identify over-partitioning before it creates tiny-file/metadata problems.

3. Prerequisite / starting state

Prerequisite: DE11-04 mastered with saved evidence.

4. Picture it in your head

Dataset partitioning places records into directory/file groups based on a partition key, often a low-to-moderate-cardinality field used for pruning. Good partitioning reduces the candidate files for common queries.

5. New words before we use them

Dataset partitioning places records into directory/file groups based on a partition key, often a low-to-moderate-cardinality field used for pruning. Good partitioning reduces the candidate files for common queries. Bad partitioning can create excessive directories and tiny files.

6. Useful video / source

Source: Databricks Academy — free data/lakehouse training

Assigned scope: Choose the matching storage / Parquet / Delta module assigned by Mentor

How to use it: Use the matching free training unit for the visual explanation, then return to the Teaching Book and execute the local course lab.

[Open source](#)

7. Follow with me once

The layout lab partitions 12 orders into four region directories. A hypothetical East-only query has one candidate region file instead of four region files. That does not prove Parquet row-group skipping; it demonstrates directory pruning. Compare region, order_date and order_id as candidate partition keys. For each, calculate distinct values on the 12-row data and explain expected file-count pressure.

8. Expected result / what you should see

Course data distinct counts: region=4, order_date=4, order_id=12. A partition-by-order_id plan creates the clearest over-partitioning warning here.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

If you cannot choose a partition key, start from the most common selective query boundaries and data volume. Never choose only because a column “looks categorical.”

11. If you get stuck

If you cannot choose a partition key, start from the most common selective query boundaries and data volume. Never choose only because a column “looks categorical.”

12. Small independent task

Design partitioning for a 500M-row event table with frequent last-7-day queries and a very high-cardinality customer_id. Defend your first choice and name the metric that would make you revisit it.

13. Validate

Course data distinct counts: region=4, order_date=4, order_id=12. A partition-by-order_id plan creates the clearest over-partitioning warning here.

14. Teach it back

Why can over-partitioning make a columnar lake slower even though partition pruning sounds beneficial?

15. Protected review + changed retry

Only after the independent attempt and evidence are saved may you open the matching M11 Review/Repair section. Close it before a changed retry.

16. Evidence / mastery + exact next route

Save the command/code or decision artifact, observed output/control, one error or uncertainty, the repair you made, and a 60-90 second explain-back. Do not mark mastered from reading alone. Exact next route: DE11-06

DE11-06 - Lake vs warehouse vs lakehouse

1. Why it matters

This matters because later Storage & File Formats Foundations work assumes this competency is reliable, not merely familiar. You must be able to reproduce it and validate the result without copying a finished answer.

2. What you are learning

Compare data lake, data warehouse and lakehouse by storage, governance, transaction and workload needs without treating one architecture as universally best.

3. Prerequisite / starting state

Prerequisite: DE11-05 mastered with saved evidence.

4. Picture it in your head

A data warehouse traditionally emphasizes managed structured tables, SQL analytics, governance and optimized query serving. A data lake emphasizes low-cost scalable object storage for many data types, but a plain collection of files does not automatically provide table transactions or strong governance.

5. New words before we use them

A data warehouse traditionally emphasizes managed structured tables, SQL analytics, governance and optimized query serving. A data lake emphasizes low-cost scalable object storage for many data types, but a plain collection of files does not automatically provide table transactions or strong governance. A lakehouse combines open/scalable lake storage with table-management capabilities such as reliable transactions, schemas, metadata and SQL-friendly access.

6. Useful video / source

Source: Databricks Academy — free data/lakehouse training

Assigned scope: Choose the matching storage / Parquet / Delta module assigned by Mentor

How to use it: Use the matching free training unit for the visual explanation, then return to the Teaching Book and execute the local course lab.

[Open source](#)

7. Follow with me once

Place the M11 retail data into three conceptual designs: raw object files, curated warehouse tables, and lakehouse tables over object storage. For each, state where transaction/table metadata lives and who enforces contracts. Complete an architecture matrix for storage cost, schema governance, BI serving, ML/file access, transaction support and operational complexity.

8. Expected result / what you should see

Your answer must name trade-offs. “Lakehouse is newer so always better” is an automatic reasoning defect.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

If the terms blur together, separate physical storage (files/object store) from table-management/catalog/query layers.

11. If you get stuck

If the terms blur together, separate physical storage (files/object store) from table-management/catalog/query layers.

12. Small independent task

Given a small company with 50 GB structured sales data and five BI users, argue both for and against introducing a lakehouse now. Make a recommendation based on need, not fashion.

13. Validate

Your answer must name trade-offs. “Lakehouse is newer so always better” is an automatic reasoning defect.

14. Teach it back

What capability turns a collection of Parquet files into something closer to a managed table?

15. Protected review + changed retry

Only after the independent attempt and evidence are saved may you open the matching M11 Review/Repair section. Close it before a changed retry.

16. Evidence / mastery + exact next route

Save the command/code or decision artifact, observed output/control, one error or uncertainty, the repair you made, and a 60-90 second explain-back. Do not mark mastered from reading alone. Exact next route: DE11-07

DE11-07 - Delta tables, transaction log and ACID concepts

1. Why it matters

This matters because later Storage & File Formats Foundations work assumes this competency is reliable, not merely familiar. You must be able to reproduce it and validate the result without copying a finished answer.

2. What you are learning

Explain Delta table fundamentals: Parquet data files plus a transaction log, atomic commits, consistent table versions, and why direct file mutation is unsafe.

3. Prerequisite / starting state

Prerequisite: DE11-06 mastered with saved evidence.

4. Picture it in your head

Delta Lake adds a transaction-log layer around data files. Instead of deciding table state by listing whatever files happen to exist, readers reconstruct a committed table version from the log.

5. New words before we use them

Delta Lake adds a transaction-log layer around data files. Instead of deciding table state by listing whatever files happen to exist, readers reconstruct a committed table version from the log. A transaction commit records which files/actions become part of the next version.

6. Useful video / source

Source: Databricks Academy — free data/lakehouse training

Assigned scope: Choose the matching storage / Parquet / Delta module assigned by Mentor

How to use it: Use the matching free training unit for the visual explanation, then return to the Teaching Book and execute the local course lab.

[Open source](#)

7. Follow with me once

Run `delta_foundation_simulator.py`. It writes version_0/1/2 snapshots plus numbered classroom log records. This is deliberately NOT a Delta engine; it exists to make versioned commit thinking visible before M13. Inspect log versions 0,1,2. Identify operation, row count and schema. Explain why a reader should move from one committed version to the next rather than combine half-written state.

8. Expected result / what you should see

Simulator versions are [0,1,2]. V0 has 4 rows / spend 820.0; V1 has 5 rows / spend 1050.0; V2 adds `segment` without changing row count.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

If you equate Delta with “Parquet plus a JSON folder,” focus on the protocol: correctness comes from committed log semantics, not merely the file names.

11. If you get stuck

If you equate Delta with “Parquet plus a JSON folder,” focus on the protocol: correctness comes from committed log semantics, not merely the file names.

12. Small independent task

Draw a failure timeline where new data files exist but the transaction commit is not published. Explain what an atomic table protocol should expose to a reader.

13. Validate

Simulator versions are [0,1,2]. V0 has 4 rows / spend 820.0; V1 has 5 rows / spend 1050.0; V2 adds `segment` without changing row count.

14. Teach it back

Why is a transaction log more important than simply having Parquet files in a folder?

15. Protected review + changed retry

Only after the independent attempt and evidence are saved may you open the matching M11 Review/Repair section. Close it before a changed retry.

16. Evidence / mastery + exact next route

Save the command/code or decision artifact, observed output/control, one error or uncertainty, the repair you made, and a 60-90 second explain-back. Do not mark mastered from reading alone. Exact next route: DE11-08

DE11-08 - MERGE/upsert and current-state records

1. Why it matters

This matters because later Storage & File Formats Foundations work assumes this competency is reliable, not merely familiar. You must be able to reproduce it and validate the result without copying a finished answer.

2. What you are learning

Reason about MERGE/upsert as matched-update plus unmatched-insert logic, prove current-state controls, and detect ambiguous source matches.

3. Prerequisite / starting state

Prerequisite: DE11-07 mastered with saved evidence.

4. Picture it in your head

An upsert combines update and insert behavior. Delta `MERGE` expresses a match condition between source and target, then applies actions for matched and unmatched rows.

5. New words before we use them

An upsert combines update and insert behavior. Delta `MERGE` expresses a match condition between source and target, then applies actions for matched and unmatched rows. The most important engineering work happens before the syntax: define the target grain and business key, then ensure the source does not contain multiple rows that ambiguously match the same target key unless your logic deliberately resolves them.

6. Useful video / source

Source: Databricks Academy — free data/lakehouse training

Assigned scope: Choose the matching storage / Parquet / Delta module assigned by Mentor

How to use it: Use the matching free training unit for the visual explanation, then return to the Teaching Book and execute the local course lab.

[Open source](#)

7. Follow with me once

Simulator v0 has C1-C4. Updates change C2 from Gold/350 to Platinum/420 and insert C5/160. After upsert, target has C1-C5 and spend 1050. Write the four controls before running: matched keys, inserted keys, expected target row count, expected total spend. Then compare to simulator report.

8. Expected result / what you should see

Expected simulator MERGE: updated=[C2], inserted=[C5], rows=5, spend=1050.0. Multiple source records for one target key must be resolved before claiming safe current-state logic.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

If target rows unexpectedly multiply, stop and test key uniqueness on both target and intended source update grain before changing the MERGE clause.

11. If you get stuck

If target rows unexpectedly multiply, stop and test key uniqueness on both target and intended source update grain before changing the MERGE clause.

12. Small independent task

Create a paper source with two different updates for C2. Explain why the source is ambiguous and propose one deterministic pre-merge rule using event timestamp + tie-breaker.

13. Validate

Expected simulator MERGE: updated=[C2], inserted=[C5], rows=5, spend=1050.0. Multiple source records for one target key must be resolved before claiming safe current-state logic.

14. Teach it back

Why can a syntactically valid MERGE still be logically unsafe?

15. Protected review + changed retry

Only after the independent attempt and evidence are saved may you open the matching M11 Review/Repair section. Close it before a changed retry.

16. Evidence / mastery + exact next route

Save the command/code or decision artifact, observed output/control, one error or uncertainty, the repair you made, and a 60-90 second explain-back. Do not mark mastered from reading alone. Exact next route: DE11-09

DE11-09 - Schema enforcement, controlled evolution and compatibility

1. Why it matters

This matters because later Storage & File Formats Foundations work assumes this competency is reliable, not merely familiar. You must be able to reproduce it and validate the result without copying a finished answer.

2. What you are learning

Distinguish schema enforcement from schema evolution and classify additive, rename/drop and type changes by compatibility risk.

3. Prerequisite / starting state

Prerequisite: DE11-08 mastered with saved evidence.

4. Picture it in your head

Schema enforcement rejects or blocks writes that violate the table's expected structure or types. Schema evolution is a controlled process for accepting intended structural change.

5. New words before we use them

Schema enforcement rejects or blocks writes that violate the table's expected structure or types. Schema evolution is a controlled process for accepting intended structural change. The safest beginner pattern is “fail closed, inspect, then evolve deliberately,” not “accept any new schema automatically.” Additive nullable columns are often easier to accommodate than incompatible type changes.

6. Useful video / source

Source: Databricks Academy — free data/lakehouse training

Assigned scope: Choose the matching storage / Parquet / Delta module assigned by Mentor

How to use it: Use the matching free training unit for the visual explanation, then return to the Teaching Book and execute the local course lab.

[Open source](#)

7. Follow with me once

Simulator v2 adds `segment` to the v1 schema. It is a controlled additive change. Existing five rows remain five rows; each gets a deterministic derived segment. Classify four hypothetical changes: add nullable promo_code, rename amount to net_amount, widen INT to DOUBLE, change date to free-text STRING. Write consumer impact and migration evidence for each.

8. Expected result / what you should see

V2 schema is [id,name,tier,spend,segment] and v2_rows remains 5. Row-count stability alone is not enough; verify schema and key/control totals too.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

If “schema evolution” sounds like permission to ignore drift, rewrite the rule as: detect -> classify -> approve -> migrate -> validate -> monitor.

11. If you get stuck

If “schema evolution” sounds like permission to ignore drift, rewrite the rule as: detect -> classify -> approve -> migrate -> validate -> monitor.

12. Small independent task

Design a schema-change approval checklist with at least one producer check, one consumer check, rollback/replay plan and ownership field.

13. Validate

V2 schema is [id,name,tier,spend,segment] and v2_rows remains 5. Row-count stability alone is not enough; verify schema and key/control totals too.

14. Teach it back

What is the difference between enforcement and evolution, and why can an additive column still break a consumer?

15. Protected review + changed retry

Only after the independent attempt and evidence are saved may you open the matching M11 Review/Repair section. Close it before a changed retry.

16. Evidence / mastery + exact next route

Save the command/code or decision artifact, observed output/control, one error or uncertainty, the repair you made, and a 60-90 second explain-back. Do not mark mastered from reading alone. Exact next route: DE11-10

DE11-10 - Time travel, retention, VACUUM safety and medallion checkpoint

1. Why it matters

This matters because later Storage & File Formats Foundations work assumes this competency is reliable, not merely familiar. You must be able to reproduce it and validate the result without copying a finished answer.

2. What you are learning

Use version history/time travel conceptually, explain retention/VACUUM risk, and defend a bronze-silver-gold medallion checkpoint without deleting evidence prematurely.

3. Prerequisite / starting state

Prerequisite: DE11-09 mastered with saved evidence.

4. Picture it in your head

Time travel means querying a prior committed table version or timestamp while its required files/log history are still retained. It is useful for debugging, audit comparison and reproducibility.

5. New words before we use them

Time travel means querying a prior committed table version or timestamp while its required files/log history are still retained. It is useful for debugging, audit comparison and reproducibility. Retention policies decide how long historical data files remain available.

6. Useful video / source

Source: Databricks Academy — free data/lakehouse training

Assigned scope: Choose the matching storage / Parquet / Delta module assigned by Mentor

How to use it: Use the matching free training unit for the visual explanation, then return to the Teaching Book and execute the local course lab.

[Open source](#)

7. Follow with me once

Simulator proves time-travel difference: C2 is Gold in v0 and Platinum in current v2. Its VACUUM action is DRY_RUN_ONLY on purpose. No historical file is deleted in this module. Write a retention decision for a daily table with 10-day incident investigations and 7-day late reprocessing. Then map the retail source through bronze/silver/gold with one validation at each boundary.

8. Expected result / what you should see

Time-travel control: v0 C2 tier=Gold; current v2 C2 tier=Platinum. VACUUM remains dry-run only. Your medallion checklist must preserve raw evidence and include row/key/control reconciliation.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

If time travel and backups seem identical, separate table-version history from an independent backup/disaster-recovery copy. They solve overlapping but different failure cases.

11. If you get stuck

If time travel and backups seem identical, separate table-version history from an independent backup/disaster-recovery copy. They solve overlapping but different failure cases.

12. Small independent task

Explain why running VACUUM aggressively immediately after every job could damage debugging/recovery. Then design a medallion promotion checklist for an unseen dataset.

13. Validate

Time-travel control: v0 C2 tier=Gold; current v2 C2 tier=Platinum. VACUUM remains dry-run only. Your medallion checklist must preserve raw evidence and include row/key/control reconciliation.

14. Teach it back

How can retention policy change the usefulness of time travel, and what must be true before bronze data is promoted to silver?

15. Protected review + changed retry

Only after the independent attempt and evidence are saved may you open the matching M11 Review/Repair section. Close it before a changed retry.

16. Evidence / mastery + exact next route

Save the command/code or decision artifact, observed output/control, one error or uncertainty, the repair you made, and a 60-90 second explain-back. Do not mark mastered from reading alone. Exact next route: Proceed to the module practical/Gate route after the module mastery checklist.

Module mastery checklist + revision quick reference

- ☐ Every lesson has a saved genuine independent attempt.
- ☐ Every lesson has at least one independent validation/control appropriate to the tool.
- ☐ Protected reviews were opened only after attempts; any weak lesson has a changed retry after review was closed.
- ☐ You can teach back each lesson in your own words and name one realistic failure mode.
- ☐ Required module practical/retest evidence is saved and no critical defect is unresolved.
- ☐ D+1, D+4, D+10 and D+30 revision events are scheduled for weak or high-value competencies.

Revision cadence

D+1: fresh micro-task. D+4: mixed retrieval. D+10: changed/failure case. D+30: interview/project retrieval. Always attempt from memory before reopening a source.

Release boundary

This book is a static learner artifact. It does not prove live tool/cloud execution or a protected Gate pass. Runtime evidence must be recorded separately when the curriculum requires it.